

AGENCYOS

UI Design & Prototype Workflow Specification

Document 12 of the AgencyOS Master Documentation Set

AI UI DESIGNER • THEMES • SCREENS • SCOPE COVERAGE • ADMIN APPROVAL • CLIENT REVIEW
• REVISIONS • UI PROTOTYPE APK/WEB

Purpose: Define the complete UI design and prototype pipeline from approved scope through AI-generated design, theme selection, screen coverage, internal validation, Admin approval, client review, revision control and a functional prototype delivered as web and/or APK where applicable.

1. Core Principle

The UI Designer Agent must design from the approved scope, not from imagination alone. Every important screen and interaction should map back to a requirement, feature, workflow or explicit design decision.

- Scope is the source of truth for what needs to be designed.
- AI proposes the visual system and screen structure.
- The system checks coverage before a design can be considered complete.
- Admin/PM approval controls internal readiness.
- Client review controls client acceptance where configured.
- Revisions are versioned.
- Approved UI becomes the reference for prototype and downstream development.
- Prototype output must correspond to the approved UI version.

2. End-to-End UI Workflow

APPROVED SCOPE → UI ANALYSIS → DESIGN SYSTEM/THEME → SCREEN INVENTORY → WIREFRAME/STRUCTURE → HIGH-FIDELITY UI → COVERAGE CHECK → INTERNAL REVIEW → ADMIN/PM APPROVAL → CLIENT REVIEW → REVISIONS → UI APPROVAL → PROTOTYPE → PROTOTYPE REVIEW → DEVELOPMENT HANDOFF

- A project may skip intermediate representations only when the configured project type permits it.
- The workflow must never silently jump from incomplete requirements to 'UI completed'.

3. UI Design Inputs

- Active scope version.
- Requirements and feature list.
- Acceptance criteria.
- Target platform(s).
- User roles/personas.
- Business workflows.
- Brand assets.
- Client references/inspiration.
- Existing product design system where applicable.
- Accessibility requirements.
- Technical constraints.
- Device/screen requirements.
- Project timeline.
- Approved UI revision allowance.

4. UI Designer Agent Responsibilities

- Analyze scope and derive UI implications.
- Create screen inventory.
- Identify user flows.
- Propose information architecture.

- Select or generate an appropriate visual direction.
- Create reusable design tokens/components.
- Design responsive states.
- Create high-fidelity screens.
- Link screens to features/requirements.
- Generate prototype interactions.
- Prepare client-review package.
- Apply approved revisions.
- Prepare developer handoff.
- Report blockers instead of fabricating missing requirements.

5. UI Design Agent Guardrails

- Do not add material product features without scope/change approval.
- Do not remove required functionality for visual simplicity.
- Do not treat placeholder screens as complete production UI.
- Do not declare completion when required screens/states are missing.
- Do not overwrite an approved version.
- Do not consume revision allowance for agency-caused corrections where policy excludes them.
- Do not expose internal prompts, API keys or private agency data in client-facing artifacts.
- Escalate ambiguous business rules.

6. Theme Selection

Theme inputs

- Client brand colors.
- Logo/brand assets.
- Industry/context.
- Target audience.
- Product personality.
- Accessibility requirements.
- Platform conventions.
- Client references.
- Existing design system.

Theme output

- Primary/secondary colors.
- Typography.
- Spacing scale.
- Border radius.
- Elevation/shadow rules.
- Iconography.
- Button/input styles.
- Card/table/modal patterns.
- Navigation patterns.

- Light/dark mode where required.
- Responsive breakpoints.

The theme should be represented as reusable design tokens rather than isolated styling choices.

7. Theme Options & Client Choice

The UI Agent may generate multiple visual directions when the project requires a design choice.

- Option A/B/C may be presented internally or to the client according to project policy.
- Each option should use the same approved functional scope so the decision is visual, not an accidental scope change.
- Client selects a direction where required.
- Selected theme becomes the active design direction.
- Alternative concepts remain historical rather than silently disappearing.
- A new visual direction after approval may be treated as a revision or change request depending on impact.

8. Screen Inventory

Each screen record should contain

- Screen ID.
- Screen name.
- User role.
- Feature/requirement mapping.
- Purpose.
- Entry point.
- Exit/next action.
- Required data.
- Actions.
- Validation.
- Empty state.
- Loading state.
- Error state.
- Success state.
- Permission/role behavior.
- Responsive behavior.
- Accessibility considerations.
- Status/version.

9. Screen Coverage Matrix

Every approved feature should map to one or more screens, flows, components, or explicit non-UI behavior.

- Requirement → Feature → Screen(s) → Interaction(s) → Prototype state → QA coverage.
- Flag features with no UI representation when UI is required.
- Flag screens with no scope/feature mapping.
- Flag missing role states.
- Flag missing error/empty/loading states.

- Flag duplicate screens.
- Flag orphaned prototype routes.

This matrix is one of the main controls preventing an AI designer from producing attractive but incomplete work.

10. User Flow Design

- Identify primary user journeys.
- Identify alternate paths.
- Identify edge cases.
- Identify permissions.
- Identify authentication states.
- Identify success/failure paths.
- Identify navigation relationships.
- Link every flow to scope/requirements.
- Represent critical flows in the prototype.

11. Design States

- Default.
- Hover/focus where applicable.
- Pressed/active.
- Disabled.
- Loading.
- Empty.
- Validation error.
- System error.
- Success.
- Permission denied.
- Offline/network failure where applicable.
- Long-content/overflow state.

A screen is not complete merely because its happy-path screenshot looks finished.

12. Responsive Design

- Desktop layout where applicable.
- Tablet layout where applicable.
- Mobile layout where applicable.
- Breakpoints.
- Fluid/responsive behavior.
- Text wrapping.
- Table/list adaptation.
- Navigation adaptation.
- Touch targets.
- Keyboard/accessibility behavior.

Platform scope determines which responsive variants are mandatory.

13. Accessibility

- Readable contrast.
- Semantic hierarchy.
- Keyboard navigation where applicable.
- Visible focus.
- Touch target sizing.
- Labels for inputs.
- Error messaging.
- Accessible states.
- Reduced-motion considerations where applicable.
- Screen-reader semantics in implementation handoff.

Accessibility requirements should be captured in the scope and verified during QA.

14. Internal UI Quality Gate

AI DESIGN OUTPUT → AUTOMATED COVERAGE CHECK → PM/UI REVIEW → FIXES →
READY_FOR_APPROVAL

- Check scope coverage.
- Check screen inventory completeness.
- Check required states.
- Check role/permission flows.
- Check consistency of design tokens.
- Check responsive coverage.
- Check prototype route coverage.
- Check obvious visual defects.
- Check naming/versioning.
- Check missing assets.
- Check unresolved placeholders.

The UI Agent cannot pass itself solely by writing a completion message.

15. Admin / PM Approval

Approval package

- UI version.
- Scope version.
- Screen coverage report.
- Theme/design system.
- Key flows.
- Known limitations.
- Revision usage.
- Prototype readiness if available.
- Open questions.
- Preview links/files.

Possible decisions

- APPROVE.
- REJECT.
- REQUEST_CHANGES.
- APPROVE_WITH_NOTES where policy allows.
- ESCALATE.

16. Client Review

INTERNAL APPROVAL → CLIENT REVIEW → CLIENT FEEDBACK → CLASSIFY FEEDBACK → REVISION OR CHANGE REQUEST → CLIENT APPROVAL

- Client receives the exact UI version under review.
- Review instructions should be clear.
- Feedback should be linked to screen/component where possible.
- Client comments are preserved.
- The system distinguishes UI feedback from new scope.
- Client approval references the exact version.
- Generic responses should not be interpreted as formal approval when explicit approval is required.

17. Client Feedback Classification

Type	Meaning	Action
Type	Meaning	Action
BUG/CORRECTION	Does not meet approved requirement	Fix; normally no scope expansion
INCLUDED_REVISION	Within configured allowance	Apply revision
CLARIFICATION	Feedback needs explanation	Clarify
SCOPE_CHANGE	New/material functionality	Change Request
DESIGN_DIRECTION	Material visual direction change	Revision or Change Request based on policy
REJECTED_REQUEST	Not accepted/feasible	Record reason

18. Revision Management

- UI version number.
- Previous version.
- Feedback/change source.
- Revision count.
- Included/free allowance used.
- Remaining allowance.
- Change request reference where applicable.
- Updated screens.
- Updated prototype.
- Approval state.
- Timestamp.
- Actor.

Every revision must clearly state what changed from the previous version.

19. UI Version Lifecycle

DRAFT → INTERNAL_REVIEW → ADMIN_APPROVAL → CLIENT_REVIEW →
CHANGES_REQUESTED → REVISED → CLIENT_APPROVAL → LOCKED/APPROVED

- Rejected versions remain historical.
- Approved versions become read-only baselines.
- A later revision does not alter the approved version.
- Development should reference the exact approved UI version.

20. Scope Coverage Before UI Approval

- All required modules represented.
- All major features represented.
- All required user roles represented.
- All major flows represented.
- All required states represented.
- All required platforms represented.
- Excluded features not accidentally designed as commitments.
- No unresolved material placeholders.
- All screens have feature/requirement mapping.
- Client-specific branding requirements satisfied.

21. Prototype Definition

The prototype is a functional representation of approved UI and key user flows. It is not automatically the final production application.

- Prototype should use approved UI version.
- Prototype should cover priority/critical flows.
- Interactions should be meaningful rather than decorative.
- Navigation should be testable.
- Inputs can be mocked where backend integration is not yet required.
- Prototype data can be sample/mock data clearly separated from production data.
- Prototype limitations should be disclosed.

22. Prototype Types

Web Prototype

- Browser-accessible preview.
- Responsive where required.
- Shareable review URL or controlled preview environment.
- Version-specific build.
- Test/reset mechanism where useful.

Mobile Prototype / APK

- Installable APK where Android prototype is required.

- Versioned package.
- Test credentials/data where necessary.
- Device compatibility target.
- Clear prototype-only limitations.

For iOS, use the configured platform-specific prototype/distribution mechanism rather than assuming APK delivery.

23. Prototype Generation

APPROVED UI → COMPONENT/SCREEN IMPLEMENTATION → ROUTING → MOCK DATA → INTERACTIONS → VALIDATION → BUILD → PREVIEW/APK → REVIEW

- Generate only approved screens/flows.
- Implement core interactions.
- Connect screen transitions.
- Add realistic representative data.
- Implement form validation where required for the flow.
- Implement loading/error/success states for important paths.
- Package web and/or mobile output.
- Record build/version metadata.

24. Prototype Quality Gate

- All critical screens load.
- All critical routes work.
- No broken links/buttons in covered flows.
- No major visual overflow.
- Required states exist.
- Approved theme is applied consistently.
- Screen coverage matches scope matrix.
- Prototype version matches UI version.
- Known limitations are documented.
- No production secrets are embedded.

Prototype completion requires actual validation, not merely successful compilation.

25. Client Prototype Review

- Send exact prototype version.
- Provide review instructions.
- Collect feedback.
- Record defects separately from new scope.
- Track revision count.
- Request explicit approval where required.
- Lock approved prototype version.

If the prototype exposes a functional misunderstanding, the system should determine whether it is a design defect, clarification or scope change.

26. UI-to-Development Handoff

Handoff package

- Approved UI version.
- Approved prototype version.
- Design tokens.
- Screen inventory.
- Component list.
- User flows.
- Assets.
- Responsive rules.
- States.
- Interaction behavior.
- Accessibility notes.
- Feature/requirement mappings.
- Acceptance criteria.
- Known limitations.
- Developer notes.

Development must start from the approved baseline, not from a developer's memory of the WhatsApp conversation.

27. Asset Management

- Logo.
- Icons.
- Images.
- Illustrations.
- Fonts/licenses where applicable.
- Favicon/app icon.
- Animations/video assets where applicable.
- Asset source/reference.
- Version.
- Usage rights/constraints where supplied.

Secrets or private credentials must never be treated as ordinary design assets.

28. UI Agent + Multi-Model Routing

- UI Agent may use the configured AI model selected by the Orchestrator.
- Model selection can consider visual capability, reasoning, cost, latency and availability.
- A fallback model can be configured.
- The UI workflow remains model-agnostic.
- Provider/model used for significant generation tasks should be logged where appropriate.
- A model failure must not corrupt the UI version.

The Orchestrator and Policy Engine govern model/provider selection; the UI Agent focuses on design work.

29. Admin Controls

- Approve/reject UI versions.
- Approve client review release.
- Configure revision allowance.
- Configure free-change thresholds.
- Pause UI automation.
- Reassign UI Agent/model.
- Select preferred visual/theme strategy.
- Lock approved version.
- View scope coverage.
- View design/prototype history.
- Approve prototype release.
- Approve developer handoff.
- Override supported workflow states with reason.
- Inspect audit trail.

30. Project Manager Agent Responsibilities

- Provide scope/context to UI Agent.
- Create UI milestone.
- Track UI progress.
- Review coverage report.
- Coordinate Admin approval.
- Coordinate client review.
- Classify feedback.
- Trigger change request when needed.
- Track revisions.
- Approve readiness for prototype.
- Coordinate prototype review.
- Prepare development handoff.

PM owns the workflow; UI Agent owns the design execution.

31. Preventing Incomplete UI

- Require screen inventory.
- Require feature mapping.
- Require critical-flow coverage.
- Require state coverage.
- Require responsive coverage where applicable.
- Require internal quality gate.
- Require explicit approval.
- Require prototype validation.
- Block handoff if mandatory coverage is missing.
- Detect orphan screens and unmapped features.
- Detect unresolved placeholders.
- Detect version mismatch between UI and prototype.

32. UI Risk & Blockers

- Missing requirement.
- Conflicting requirement.
- Missing asset.
- Brand ambiguity.
- Unsupported interaction.
- Platform limitation.
- Accessibility conflict.
- Prototype implementation limitation.
- Client feedback ambiguity.
- Scope change.
- Model generation failure.

Blockers should be structured and routed to PM/Admin rather than hidden inside an agent's output.

33. Design & Prototype Audit

- Who created version.
- Model/provider used where applicable.
- Source scope version.
- Source requirement version.
- Revision history.
- Admin approval.
- Client review.
- Client approval.
- Prototype build ID/version.
- Build timestamp.
- Handoff timestamp.
- Change request references.
- Known limitations.

Audit data should allow AgencyOS to answer: 'Which UI did the client approve, and what scope did it represent?'

34. UI Dashboard

- Current UI version.
- Current approval state.
- Screen count.
- Mapped feature count.
- Unmapped feature count.
- Critical flow coverage.
- Revision usage.
- Open feedback.
- Change requests.
- Prototype status.
- Prototype build version.

- Client approval status.
- Developer handoff status.
- UI blockers.

35. Example Workflow

Scope V1 approved → UI Agent extracts 18 required screens → creates theme + design system → designs screens → coverage check passes → PM/Admin approves UI V1 → client reviews → client requests 3 minor changes → changes fit revision allowance → UI V2 → client approves → prototype V2 generated → critical flows tested → client approves prototype → development handoff.

If the client instead requests a new payment workflow not present in Scope V1, the system creates a Change Request rather than quietly adding screens and backend behavior.

36. Acceptance Criteria

- UI Agent receives active scope and requirement context.
- UI theme/design system is structured and reusable.
- Screen inventory is generated.
- Every required feature can be mapped to UI where applicable.
- Critical user flows are represented.
- Loading/error/empty/success states are covered where required.
- Responsive behavior is defined for applicable platforms.
- Accessibility requirements are represented.
- Internal UI quality checks exist.
- Admin/PM approval is explicit.
- Client review references an exact UI version.
- Client feedback is classified.
- Revisions are versioned and counted.
- Free revisions are policy-controlled.
- Material changes become Change Requests.
- Approved UI is locked as a baseline.
- Prototype references the approved UI version.
- Web prototype can be generated where required.
- Android APK prototype can be generated where required.
- Prototype interactions are actually tested.
- Prototype limitations are documented.
- Development handoff contains the approved baseline.
- UI/prototype history is auditable.
- Agents cannot declare UI complete with missing mandatory coverage.

37. Final Architecture

APPROVED SCOPE → AI UI DESIGNER → THEME/DESIGN SYSTEM → SCREEN INVENTORY → FEATURE COVERAGE → UI DESIGN → INTERNAL QA → ADMIN/PM APPROVAL → CLIENT REVIEW → REVISION/CHANGE REQUEST → UI APPROVAL → PROTOTYPE WEB/APK → PROTOTYPE QA → CLIENT APPROVAL → DEVELOPMENT HANDOFF

The UI system is designed to make the AI Designer autonomous without making it careless. It must know what it is designing, prove that the scope is covered, preserve versions, respect revision limits, obtain the required approvals and produce a prototype that actually represents the approved product.

The key outcome: before coding begins, AgencyOS should know exactly which UI version is approved, which scope it represents, which screens and flows are covered, which revisions were consumed, and which prototype build the development team must use as its baseline.